

BÀI 1. MÃ HÓA ĐỐI XỨNG CỔ ĐIỂN

1.1 MẬT MÃ CEASAR

1.1.1 Giới thiệu:

Mật mã Caesar, còn được gọi là mã hoá dịch chuyển, là một phương pháp đơn giản để mã hoá thông điệp bằng cách dịch chuyển các ký tự trong bản rõ (plaintext) sang các ký tự khác trong bản mã (ciphertext). Phương pháp này được đặt tên theo Julius Caesar, nhà lãnh đạo quân đội La Mã cổ đại, người được cho là đã sử dụng nó để giao tiếp bí mật.

Cách thức hoạt động của mật mã Caesar rất đơn giản. Mỗi ký tự trong bản rõ được dịch chuyển một số lượng cố định các vị trí theo bảng chữ cái. Ví dụ, nếu chúng ta sử dụng một dịch chuyển (shift) của 3, "A" sẽ được mã hoá thành "D", "B" thành "E", và tiếp tục như vậy. Quá trình giải mã được thực hiện bằng cách dịch ngược lại.

Ví dụ minh họa với một thông điệp và một dịch chuyển là 3:

- + Bản rõ (Plaintext): HELLO
- + Bản mã (Ciphertext): KHOOR

Để giải mã, chúng ta sẽ dịch ngược lại các ký tự:

- + Bản mã (Ciphertext): KHOOR
- + Bản rõ (Plaintext): HELLO

Mã hóa cũng có thể được biểu diễn thông qua số học mô-đun, bằng cách gán các ký tự bằng các con số, theo tuần tự, $A \rightarrow 0, B \rightarrow 1, \dots, Z \rightarrow 25$. Mã hóa một chữ cái x bằng phép dịch chuyển n vị trí có thể mô tả bằng biểu thức toán học sau: $E_n(x) = (x + n) \bmod 26$. Giải mã được mô tả tương tự: $D_n(x) = (x - n) \bmod 26$.

Mật mã Caesar có nhược điểm là rất dễ bị giải mã bằng cách thử từng phương án dịch chuyển vì trong bảng chữ cái tiếng Anh chỉ có 26 chữ cái.

Bản rõ	T	O	I	Y	E	U	V	I	E	T	N	A	M
i	19	14	8	23	4	20	21	8	4	19	13	0	12
$(i+k) \bmod N$	23	18	12	1	8	24	25	12	8	23	17	4	16
Bản mã	Y	S	M	B	Y	Z	W	M	I	Y	R	E	Q

1.1.2 Bài tập thực hành:

Chuẩn bị môi trường thực hành:

- JDK (Java Development Kit): Là một bộ công cụ phát triển cho ngôn ngữ lập trình Java, được phát triển bởi Oracle Corporation. JDK cung cấp các công cụ cần thiết để phát triển, biên dịch, gỡ lỗi và thực thi ứng dụng Java. Đây là một phần mềm miễn phí và mã nguồn mở.
- Tải về và cài đặt JDK tại: <https://www.oracle.com/java/technologies/downloads/>

ORACLE Products Industries Resources Customers Partners Developers Company

Java downloads Tools and resources Java archive

JDK 22 JDK 21 JDK 17 GraalVM for JDK 22 GraalVM for JDK 21 GraalVM for JDK 17

JDK Development Kit 22.0.1 downloads

JDK 22 binaries are free to use in production and free to redistribute, at no cost, under the [Oracle No-Fee Terms and Conditions \(NFTC\)](#).

JDK 22 will receive updates under these terms, until September 2024, when it will be superseded by JDK 23.

Linux macOS **Windows**

Product/file description	File size	Download
x64 Compressed Archive	184.14 MB	https://download.oracle.com/java/22/latest/jdk-22_windows-x64_bin.zip (sha256)
x64 Installer	164.31 MB	https://download.oracle.com/java/22/latest/jdk-22_windows-x64_bin.exe (sha256)
x64 MSI Installer	163.06 MB	https://download.oracle.com/java/22/latest/jdk-22_windows-x64_bin.msi (sha256)

- Chọn phiên bản phù hợp với hệ điều hành đang sử dụng.
- Sau khi tải về JDK, tiến hành cài đặt vào máy tính như các ứng dụng bình thường khác. Kiểm tra JDK đã được cài đặt chưa bằng cách vào cửa sổ Command Prompt và gõ lệnh "java --version". Nếu có hiển thị một phiên bản Java thì JDK đã được cài đặt thành công.
- Apache Netbeans IDE: Apache NetBeans IDE là một môi trường phát triển tích hợp (Integrated Development Environment - IDE) mạnh mẽ và mã nguồn mở, được phát

triển bởi Apache Software Foundation. Đây là một công cụ phát triển phổ biến cho các nhà phát triển Java, PHP, và các ngôn ngữ lập trình khác, cung cấp nhiều tính năng hỗ trợ trong quá trình phát triển ứng dụng.

- Tải về Apache Netbeans IDE (sau đây gọi tắt là Netbeans):

<https://netbeans.apache.org/front/main/download/>

Downloading Apache NetBeans 22

Apache NetBeans 22 was released on May 29, 2024.

Apache NetBeans 22 is available for download from your closest Apache mirror.

Binaries (Platform Independent):

- [netbeans-22-bin.zip](#) (SHA-512, PGP ASC)

Installers and Packages:

- [Apache-NetBeans-22-bin-windows-x64.exe](#) (SHA-512, PGP ASC)
- [Apache-NetBeans-22.pkg](#) (SHA-512, PGP ASC)
- [apache-netbeans_22-1_all.deb](#) (SHA-512, PGP ASC)
- [apache-netbeans-22-0.noarch.rpm](#) (SHA-512, PGP ASC)
- [Linux snap package](#)

Source:

- [netbeans-22-source.zip](#) (SHA-512, PGP ASC)

Officially, it is important that you [verify the integrity](#) of the downloaded files using the PGP signatures (.asc file) or a hash (.sha512 files). The PGP keys used to sign this release are available [here](#).

Release Notes:

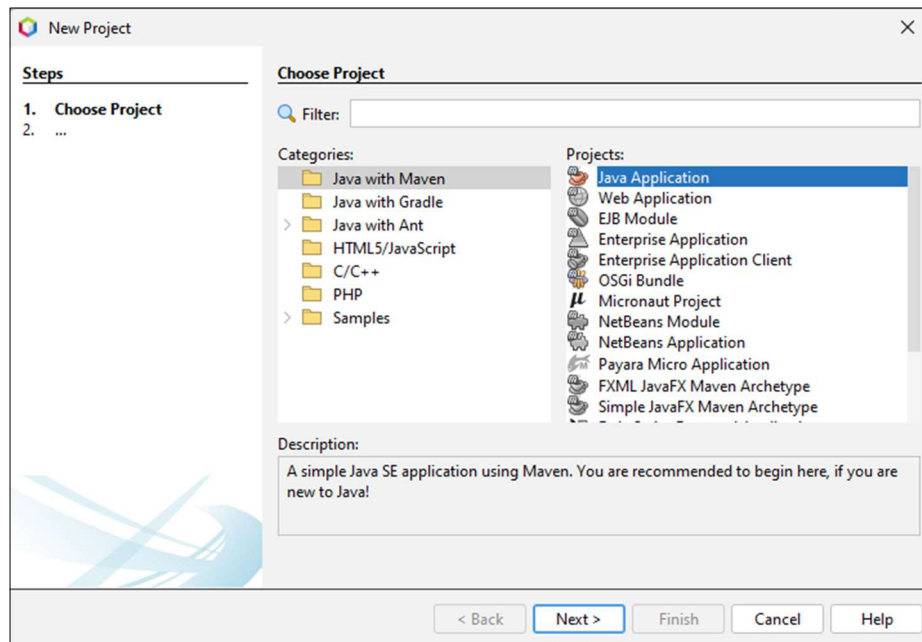
- [Github Link](#)

- [Community Installers](#)
- [Deployment Platforms](#)
- [Known Issues](#)
- [Building from Source](#)
- [Community Approval](#)
- [Earlier Releases](#)

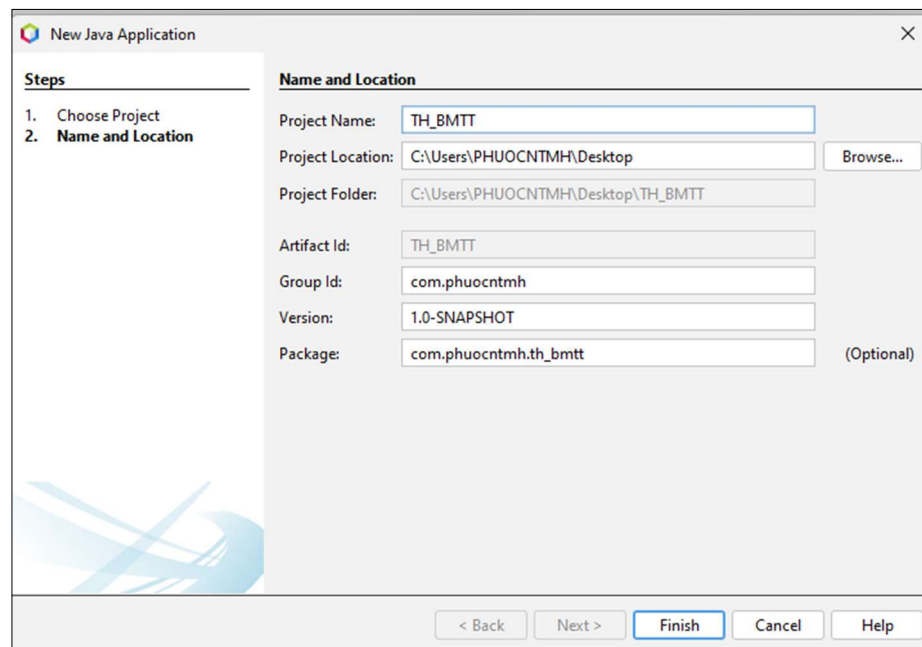
- Chọn phiên bản phù hợp với hệ điều hành đang sử dụng.
- Sau khi tải về Netbeans, tiến hành cài đặt vào máy tính như các ứng dụng bình thường khác.

Tạo ứng dụng Java với Netbeans:

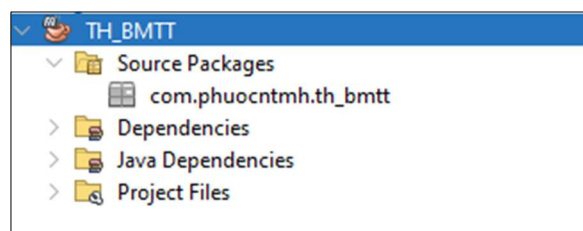
- Mở Netbeans → Chọn "File" → "New Project". Tại cửa sổ xuất hiện, chọn loại là "Java with Maven" → "Java Application". Sau đó bấm "Next" để đến bước tiếp theo.



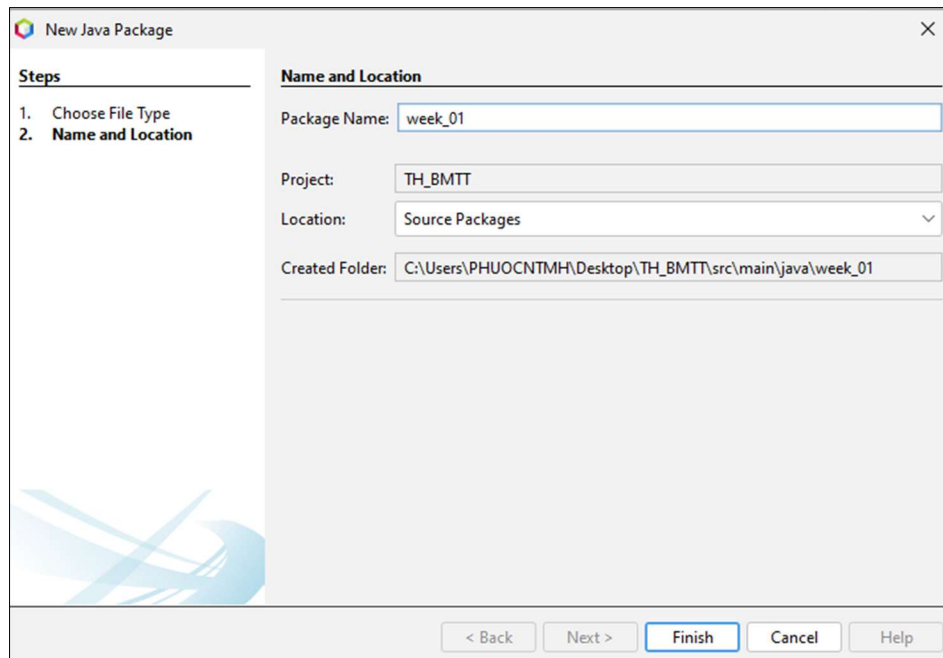
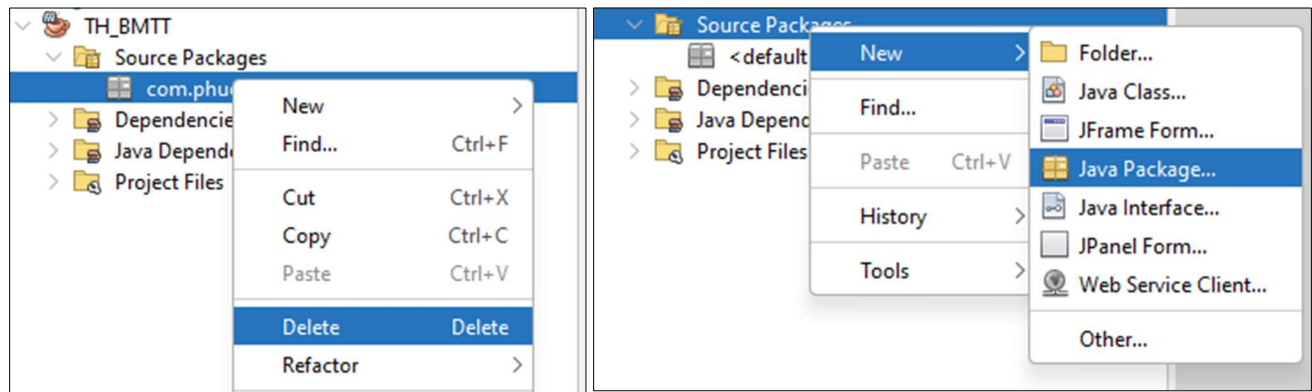
- Tại cửa sổ tiếp theo, tiến hành đặt tên cho dự án, chọn nơi lưu và điền các thông tin khác. Sau đó bấm “Finish” để tạo dự án.



- Cấu trúc của một dự án Maven như sau:



- Tiến hành xóa gói (package) mặc định của dự án. Tạo mới gói week_01”.



Thực hành với Mật mã Caesar:

- Trong package "week_01" tạo class "CaesarCipher.java".

```
package week_01;

public class CaesarCipher {

    public static String encrypt(String text, int key) {
        return caesarCipher(text, key, true);
    }

    public static String decrypt(String text, int key) {
        return caesarCipher(text, key, false);
    }
}
```

```

private static String caesarCipher(String text, int key, boolean encrypt) {
    StringBuilder result = new StringBuilder();
    int shift = encrypt ? key : -key;

    for (char character : text.toCharArray()) {
        if (Character.isLetter(character)) {
            char base = Character.isUpperCase(character) ? 'A' : 'a';
            int offset = (character - base + shift) % 26;
            if (offset < 0) {
                offset += 26;
            }
            result.append((char) (base + offset));
        } else {
            result.append(character);
        }
    }
    return result.toString();
}
}

```

- Tạo mới JFrame Form “frm_Caesar.java”:

The image shows a Java Swing window titled "Caesar Cipher Demo". Inside the window, there are three text input fields labeled "Plaintext:", "Key:", and "Ciphertext:". Below these fields, there are three buttons: "Encrypt", "Decrypt", and "Open Ciphertext File". The window has a light gray background and a standard Java Swing title bar.

- Thêm các package cần thiết:

```

package week_01;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import javax.swing.JFileChooser;
import javax.swing.JOptionPane;
import javax.swing.filechooser.FileNameExtensionFilter;

```

- Phương thức xử lý cho sự kiện nhấn nút “Encrypt”:

```

private void btn_encryptActionPerformed(java.awt.event.ActionEvent evt) {
    try {
        String plaintext = txt_plaintext.getText();
        int key = Integer.parseInt(txt_key.getText());
        String ciphertext = CaesarCipher.encrypt(plaintext, key);
        JOptionPane.showMessageDialog(this, "Encrypted successfully.");
        txt_ciphertext.setText(ciphertext);
        saveToFile(ciphertext);
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this,
            "Invalid key for Caesar Cipher. Please enter a valid integer.");
    }
}

```

- Phương thức xử lý cho sự kiện nhấn nút “Decrypt”:

```

private void btn_decryptActionPerformed(java.awt.event.ActionEvent evt) {
    try {
        String ciphertext = txt_ciphertext.getText();
        int key = Integer.parseInt(txt_key.getText());
        String plaintext = CaesarCipher.decrypt(ciphertext, key);
        JOptionPane.showMessageDialog(this, "Decrypted successfully.");
        txt_plaintext.setText(plaintext);
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this,
            "Invalid key for Caesar Cipher. Please enter a valid integer.");
    }
}

```

- Phương thức xử lý cho sự kiện nhấn nút “Open Ciphertext File”:

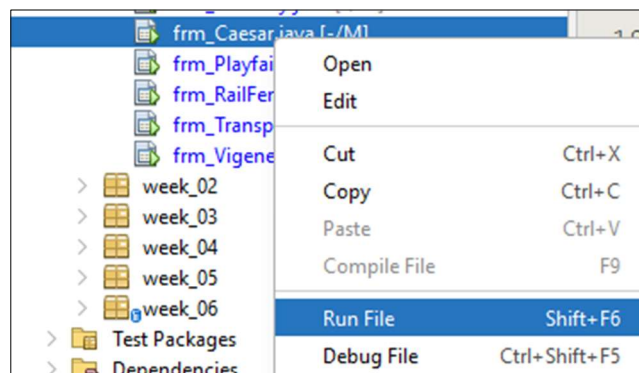

```
private void btn_openfileActionPerformed(java.awt.event.ActionEvent evt) {
    JFileChooser fileChooser = new JFileChooser();
    fileChooser.setFileFilter(new FileNameExtensionFilter("Text Files", "txt"));

    int userSelection = fileChooser.showOpenDialog(this);
    if (userSelection == JFileChooser.APPROVE_OPTION) {
        try (BufferedReader bufferedReader =
            new BufferedReader(new FileReader(fileChooser.getSelectedFile()))) {
            JOptionPane.showMessageDialog(this, "File opened successfully.");
            txt_ciphertext.read(bufferedReader, null);
        } catch (IOException e) {
            JOptionPane.showMessageDialog(this, "Error opening file: " + e.getMessage());
        }
    }
}
}
```

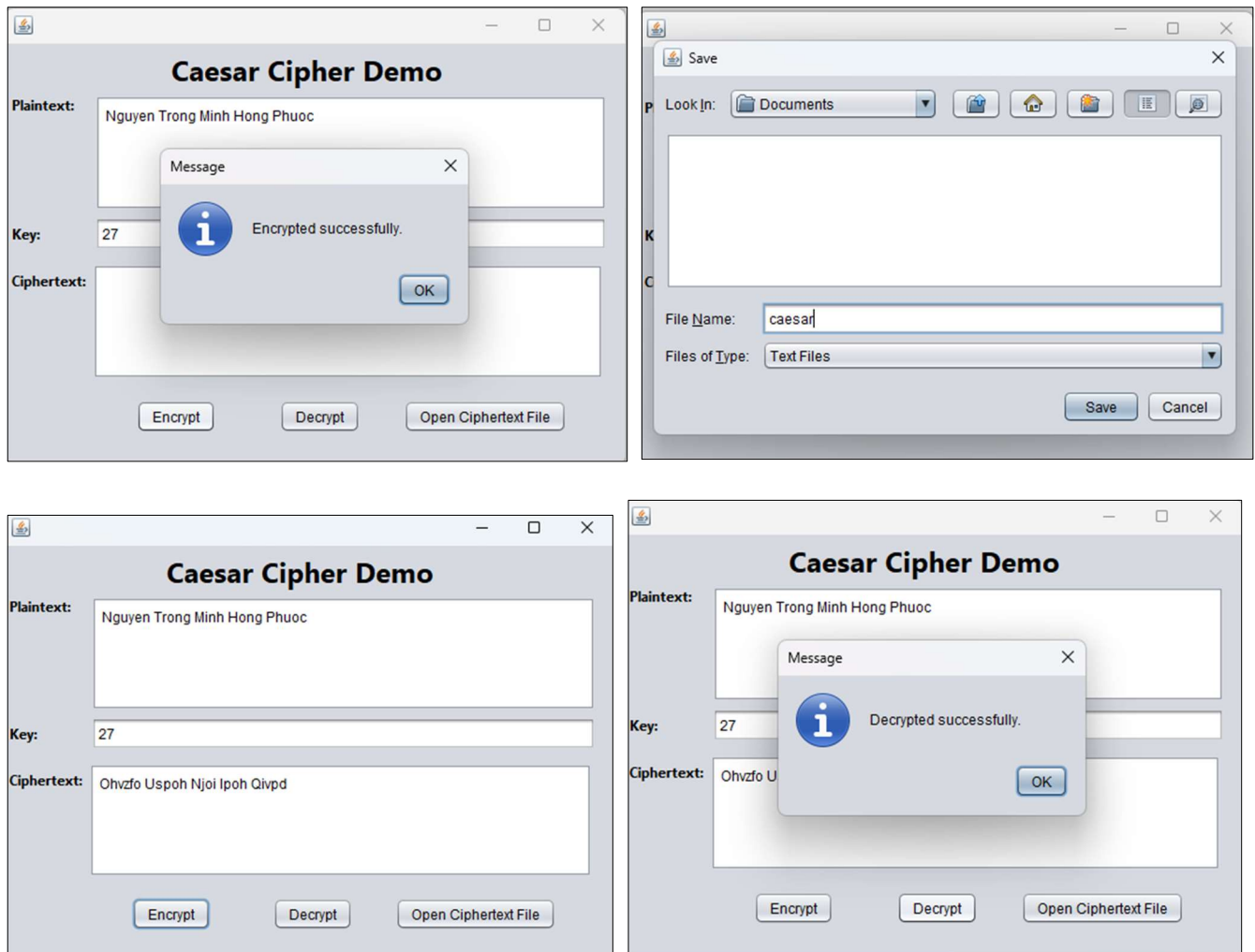
- Phương thức xử lý lưu bản mã vào tệp tin:

```
private void saveToFile(String content) {
    JFileChooser fileChooser = new JFileChooser();
    FileNameExtensionFilter filter =
        new FileNameExtensionFilter("Text Files", "txt");
    fileChooser.setFileFilter(filter);
    int userSelection = fileChooser.showSaveDialog(this);
    if (userSelection == JFileChooser.APPROVE_OPTION) {
        try (FileWriter fileWriter =
            new FileWriter(fileChooser.getSelectedFile() + ".txt")) {
            JOptionPane.showMessageDialog(this, "File saved successfully.");
            fileWriter.write(content);
        } catch (IOException e) {
            JOptionPane.showMessageDialog(this,
                "Error saving file: " + e.getMessage());
        }
    }
}
}
```

- Tiến hành chạy ứng dụng: Chuột phải tại file cần chạy, chọn "Run File" hoặc nhấn tổ hợp phím "Shift + F6".



- Kiểm tra lần lượt các nút "Encrypt", "Decrypt", "Open Ciphertext File". Khi mã hoá thành công sẽ xuất hiện hộp thoại để lưu file chứa bản mã. Tiến hành lưu file, sau đó mở file lên để nhập bản mã từ file vào JTextArea txt_ciphertext.



1.2 MẬT MÃ VIGENÈRE

1.2.1 Giới thiệu:

Mật mã Vigenère là một phương pháp mã hóa chữ cái thông điệp ban đầu bằng cách sử dụng một chuỗi khóa (keyword) để thực hiện các phép dịch chuyển trên các ký tự trong thông điệp. Phương pháp này được phát triển bởi Blaise de Vigenère vào thế kỷ 16 và được coi là một cải tiến đáng kể so với mật mã Caesar.

Cách thức hoạt động của mật mã Vigenère khá phức tạp hơn so với mật mã Caesar. Thay vì sử dụng một dịch chuyển cố định cho tất cả các ký tự trong thông điệp, mật mã Vigenère sử dụng một chuỗi khóa (keyword) để xác định các số lượng dịch chuyển khác nhau cho từng ký tự trong thông điệp.

Quy trình mã hoá thông điệp bằng mật mã Vigenère bắt đầu với việc lặp lại chuỗi khóa đến khi có độ dài bằng hoặc lớn hơn độ dài của thông điệp. Sau đó, mỗi ký tự trong chuỗi khóa được ánh xạ tương ứng với một số trong bảng chữ cái (thông thường từ A đến Z) và sẽ được sử dụng để dịch chuyển ký tự tương ứng trong thông điệp ban đầu. Ví dụ: Giả sử chúng ta có một thông điệp là "HELLO" và chuỗi khóa là "KEY". Ta thực hiện mã hoá như sau:

- + Thông điệp: H E L L O
- + Chuỗi khóa: K E Y K E
- + Ký tự mã hoá: V I Q P U

Do đó, thông điệp "HELLO" sau khi được mã hoá bằng mật mã Vigenère với chuỗi khóa "KEY" sẽ trở thành "VIQPU".

Để giải mã thông điệp được mã hoá bằng mật mã Vigenère, người nhận cần biết chuỗi khóa ban đầu để thực hiện các phép dịch chuyển ngược lại từ ký tự mã hoá để khôi phục lại thông điệp ban đầu.

1.2.2 Bài tập thực hành:

– Trong package "week_01" tạo class "VigenereCipher.java":

```
package week_01;

public class VigenereCipher {

    public static String encrypt(String text, String key) {
        return vigenereCipher(text, key, true);
    }

    public static String decrypt(String text, String key) {
        return vigenereCipher(text, key, false);
    }

    private static String vigenereCipher(String text, String key, boolean encrypt) {
        StringBuilder result = new StringBuilder();
        key = key.toLowerCase();
        int keyLength = key.length();
        int keyIndex = 0;
```

```

    for (char character : text.toCharArray()) {
        if (Character.isLetter(character)) {
            char base = Character.isUpperCase(character) ? 'A' : 'a';
            int shift = key.charAt(keyIndex % keyLength) - 'a';
            if (!encrypt) {
                shift = 26 - shift;
            }
            result.append((char) ((character - base + shift) % 26 + base));
            keyIndex++;
        } else {
            result.append(character);
        }
    }
    return result.toString();
}
}

```

- Tạo mới JFrame Form "frm_Vigenere.java":

The image shows a Java Swing window titled "Vigenère Cipher Demo". Inside the window, there are three text input fields labeled "Plaintext:", "Key:", and "Ciphertext:". Below these fields, there are three buttons: "Encrypt", "Decrypt", and "Open Ciphertext File". The window has a standard Mac OS-style title bar with a red, yellow, and green button on the left.

- Thêm các gói cần thiết:

```

package week_01;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import javax.swing.JFileChooser;
import javax.swing.JOptionPane;
import javax.swing.filechooser.FileNameExtensionFilter;

```

- Phương thức xử lý sự kiện nhấn nút "Encrypt":

```
private void btn_encryptActionPerformed(java.awt.event.ActionEvent evt) {
    String plaintext = txt_plaintext.getText();
    String key = txt_key.getText();
    String ciphertext = VigenereCipher.encrypt(plaintext, key);
    JOptionPane.showMessageDialog(this, "Encrypted successfully.");
    txt_ciphertext.setText(ciphertext);
    saveToFile(ciphertext);
}
```

- Phương thức xử lý sự kiện nhấn nút "Decrypt":

```
private void btn_decryptActionPerformed(java.awt.event.ActionEvent evt) {
    String ciphertext = txt_ciphertext.getText();
    String key = txt_key.getText();
    String plaintext = VigenereCipher.decrypt(ciphertext, key);
    JOptionPane.showMessageDialog(this, "Decrypted successfully.");
    txt_plaintext.setText(plaintext);
}
```

- Phương thức xử lý sự kiện nhấn nút "Open Ciphertext File":

```
private void btn_openfileActionPerformed(java.awt.event.ActionEvent evt) {
    JFileChooser fileChooser = new JFileChooser();
    fileChooser.setFileFilter(new FileNameExtensionFilter("Text Files", "txt"));

    int userSelection = fileChooser.showOpenDialog(this);
    if (userSelection == JFileChooser.APPROVE_OPTION) {
        try (BufferedReader bufferedReader =
            new BufferedReader(new FileReader(fileChooser.getSelectedFile()))) {
            JOptionPane.showMessageDialog(this,
                "File opened successfully.");
            txt_ciphertext.read(bufferedReader, null);
        } catch (IOException e) {
            JOptionPane.showMessageDialog(this,
                "Error opening file: " + e.getMessage());
        }
    }
}
```

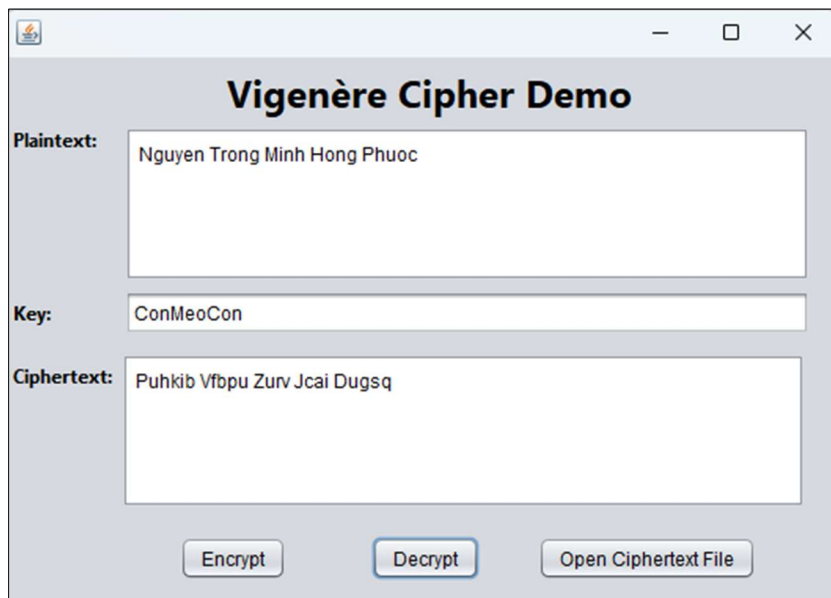
- Phương thức xử lý lưu bản mã vào file:

```

private void saveToFile(String content) {
    JFileChooser fileChooser = new JFileChooser();
    FileNameExtensionFilter filter =
        new FileNameExtensionFilter("Text Files", "txt");
    fileChooser.setFileFilter(filter);
    int userSelection = fileChooser.showSaveDialog(this);
    if (userSelection == JFileChooser.APPROVE_OPTION) {
        try (FileWriter fileWriter =
            new FileWriter(fileChooser.getSelectedFile() + ".txt")) {
            fileWriter.write(content);
        } catch (IOException e) {
            JOptionPane.showMessageDialog(this,
                "Error saving file: " + e.getMessage());
        }
    }
}
}

```

- Kiểm tra ứng dụng: Chuột phải tại file cần chạy, chọn "Run File" hoặc nhấn tổ hợp phím "Shift + F6".
- Kiểm tra lần lượt các nút "Encrypt", "Decrypt", "Open Ciphertext File". Khi mã hoá thành công sẽ xuất hiện hộp thoại để lưu file chứa bản mã. Tiến hành lưu file, sau đó mở file lên để nhập bản mã từ file vào JTextArea txt_ciphertext.



1.3 MẬT MÃ RAILFENCE

1.3.1 Giới thiệu:

Mật mã Rail Fence, hay còn được gọi là mật mã Zig-zag, là một phương pháp đơn giản để mã hoá thông điệp bằng cách viết các ký tự của thông điệp theo một mô hình đặc biệt trên các "rào cò" hoặc "rào hàng rào".

Cách thức hoạt động của mật mã Rail Fence rất đơn giản. Đầu tiên, người gửi chọn một số hàng (thường là 2 hoặc nhiều hơn) để tạo ra các "rào". Sau đó, người gửi viết các ký tự của thông điệp theo đường zigzag trên các rào. Khi đã viết hết thông điệp, người gửi sẽ đọc các ký tự theo thứ tự từ trên xuống dưới để tạo ra bản mã. Ví dụ: Giả sử chúng ta có một thông điệp là "HELLO WORLD" và chọn 3 hàng để tạo ra rào:

H	.	.	.	O	.	.	.	R	.	.	.
.	E	.	L	.	—	.	O	.	L	.	.
.	.	L	.	.	.	W	.	.	.	D	.

Khi viết các ký tự của thông điệp theo mô hình zigzag trên các rào, chúng ta thu được bản mã là "HOREL OLLWD".

Để giải mã thông điệp bằng mật mã Rail Fence, người nhận cũng cần biết số hàng đã được chọn trước đó. Người nhận sẽ tạo ra các rào tương tự và viết các ký tự của thông điệp mã hoá vào các vị trí tương ứng trên các rào. Sau đó, đọc các ký tự theo thứ tự từ trên xuống dưới để khôi phục lại thông điệp ban đầu. Mật mã Rail Fence rất dễ hiểu và triển khai, tuy nhiên, độ bảo mật của nó không cao.

1.3.2 Bài tập thực hành:

- Trong package "week_01" tạo class "RailFenceCipher.java":


```

package week_01;

public class RailFenceCipher {

    public static String encrypt(String text, int key) {
        if (key <= 1) {
            return text;
        }

        char[] chars = text.toCharArray();
        char[] result = new char[chars.length];
        int cycle = 2 * key - 2;
        int idx = 0;

        for (int i = 0; i < key; i++) {
            for (int j = 0; i + j < chars.length; j += cycle) {
                result[idx++] = chars[i + j];
                if (i != 0 && i != key - 1 && j + cycle - i < chars.length) {
                    result[idx++] = chars[j + cycle - i];
                }
            }
        }

        return new String(result);
    }
}

```

```

    public static String decrypt(String text, int key) {
        if (key <= 1) {
            return text;
        }

        char[] chars = text.toCharArray();
        char[] result = new char[chars.length];
        int cycle = 2 * key - 2;
        int idx = 0;

        for (int i = 0; i < key; i++) {
            for (int j = 0; i + j < chars.length; j += cycle) {
                result[i + j] = chars[idx++];
                if (i != 0 && i != key - 1 && j + cycle - i < chars.length) {
                    result[j + cycle - i] = chars[idx++];
                }
            }
        }

        return new String(result);
    }
}

```

- Tạo JFrame Form "frm_RailFence.java":

RailFence Cipher Demo

Plaintext:

Key:

Ciphertext:

- Thêm các gói cần thiết:

```
package week_01;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import javax.swing.JFileChooser;
import javax.swing.JOptionPane;
import javax.swing.filechooser.FileNameExtensionFilter;
```

- Phương thức xử lý sự kiện nhấn nút "Encrypt":

```
private void btn_encryptActionPerformed(java.awt.event.ActionEvent evt) {
    try {
        String plaintext = txt_plaintext.getText();
        int key = Integer.parseInt(txt_key.getText());
        String ciphertext = RailFenceCipher.encrypt(plaintext, key);
        JOptionPane.showMessageDialog(this, "Encrypted successfully.");
        txt_ciphertext.setText(ciphertext);
        saveToFile(ciphertext);
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this,
            "Invalid key for Rail Fence Cipher. Please enter a valid integer.");
    }
}
```

- Phương thức xử lý sự kiện nhấn nút "Decrypt":

```
private void btn_decryptActionPerformed(java.awt.event.ActionEvent evt) {
    try {
        String ciphertext = txt_ciphertext.getText();
        int key = Integer.parseInt(txt_key.getText());
        String plaintext = RailFenceCipher.decrypt(ciphertext, key);
        JOptionPane.showMessageDialog(this, "Decrypted successfully.");
        txt_plaintext.setText(plaintext);
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this,
            "Invalid key for Rail Fence Cipher. Please enter a valid integer.");
    }
}
```

- Phương thức xử lý sự kiện nhấn nút “Open Ciphertext File”:

```
private void btn_openfileActionPerformed(java.awt.event.ActionEvent evt) {
    JFileChooser fileChooser = new JFileChooser();
    fileChooser.setFileFilter(new FileNameExtensionFilter("Text Files", "txt"));

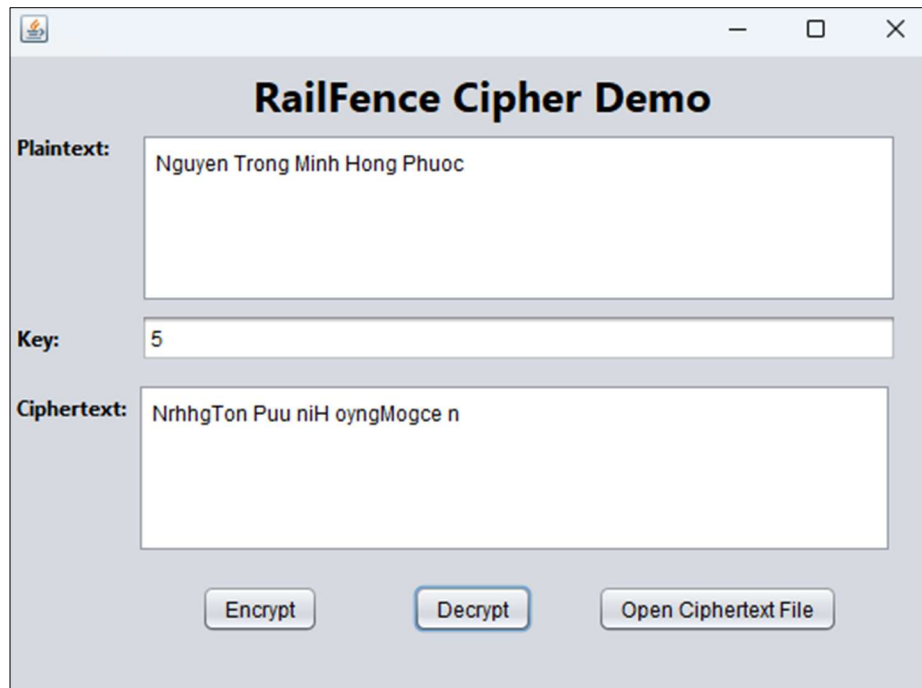
    int userSelection = fileChooser.showOpenDialog(this);
    if (userSelection == JFileChooser.APPROVE_OPTION) {
        try (BufferedReader bufferedReader =
            new BufferedReader(new FileReader(fileChooser.getSelectedFile()))) {
            JOptionPane.showMessageDialog(this, "File opened successfully.");
            txt_ciphertext.read(bufferedReader, null);
        } catch (IOException e) {
            JOptionPane.showMessageDialog(this, "Error opening file: " + e.getMessage());
        }
    }
}
```

- Phương thức xử lý lưu bản mã vào file:

```
private void saveToFile(String content) {
    JFileChooser fileChooser = new JFileChooser();
    FileNameExtensionFilter filter = new FileNameExtensionFilter("Text Files", "txt");
    fileChooser.setFileFilter(filter);
    int userSelection = fileChooser.showSaveDialog(this);
    if (userSelection == JFileChooser.APPROVE_OPTION) {
        try (FileWriter fileWriter =
            new FileWriter(fileChooser.getSelectedFile() + ".txt")) {
            JOptionPane.showMessageDialog(this, "File saved successfully.");
            fileWriter.write(content);
        } catch (IOException e) {
            JOptionPane.showMessageDialog(this, "Error saving file: " + e.getMessage());
        }
    }
}
```

- Kiểm tra ứng dụng: Chuột phải tại file cần chạy, chọn “Run File” hoặc nhấn tổ hợp phím “Shift + F6”.

- Kiểm tra lần lượt các nút “Encrypt”, “Decrypt”, “Open Ciphertext File”. Khi mã hoá thành công sẽ xuất hiện hộp thoại để lưu file chứa bản mã. Tiến hành lưu file, sau đó mở file lên để nhập bản mã từ file vào JTextArea txt_ciphertext.



1.4 MẬT MÃ PLAYFAIR

1.4.1 Giới thiệu:

Mật mã Playfair là một phương pháp mã hóa chữ cái trong một thông điệp bằng cách sử dụng một ma trận 5x5 chứa các ký tự từ bảng chữ cái. Phương pháp này đã được Charles Wheatstone đề xuất, sau đó được Herbert O. Playfair phát triển và đưa ra các quy tắc cụ thể.

Bảng Playfair thường được tạo ra bằng cách sắp xếp các ký tự từ bảng chữ cái tiếng Anh vào một ma trận 5x5, loại bỏ các ký tự trùng lặp và thường bổ sung một ký tự đặc biệt (thường là "J") nếu cần thiết để đảm bảo rằng ma trận có đủ 25 ký tự. Một khi ma trận đã được tạo, người gửi và người nhận cần thống nhất trước ma trận Playfair này để mã hoá và giải mã thông điệp. Cách thức mã hoá bằng mật mã Playfair như sau:

- Chia thông điệp cần mã hoá thành các cặp ký tự (nguyên tắc này cần một quy tắc xử lý đặc biệt nếu có số lẻ ký tự).
- Dùng các quy tắc sau để mã hoá từng cặp ký tự:

- + Nếu hai ký tự nằm trên cùng một hàng trong ma trận Playfair, thì sẽ lấy ký tự ở bên phải của mỗi ký tự (có thể trở lại đầu hàng nếu nó nằm ở cuối hàng).
- + Nếu hai ký tự nằm trên cùng một cột trong ma trận Playfair, thì sẽ lấy ký tự ở dưới mỗi ký tự (có thể quay trở lại đầu cột nếu nó nằm ở cuối cột).
- + Nếu hai ký tự không nằm trên cùng một hàng hoặc cột, ta sẽ tạo một hình chữ nhật với hai ký tự đó trong ma trận Playfair và chọn ký tự ở góc đường chéo còn lại của hình chữ nhật đó.

Để giải mã, người nhận thực hiện ngược lại quy trình trên ma trận Playfair.

Ví dụ: Giả sử chúng ta có thông điệp cần mã hoá là "HELLOWORLD". Bảng Playfair thông thường bắt đầu với việc tạo ma trận từ khóa, sau đó điền vào các ký tự còn lại từ bảng chữ cái, loại bỏ "J" nếu khóa chứa ký tự "J". Trong ví dụ này, chúng ta sẽ sử dụng khóa "KEYWORD" để tạo ma trận Playfair:

- Tạo ma trận Playfair từ khóa "KEYWORD":

K	E	Y	W	O
R	D	A	B	C
F	G	H	I/J	L
M	N	P	Q	S
T	U	V	X	Z

- Chia thông điệp thành các cặp ký tự (không cần xử lý đặc biệt cho ký tự trùng lặp): HELLOWORLD → HE LL OW OR LD.
- Xử lý đối với các cặp chứa cùng 1 ký tự (thêm pad character "X"): LL → LX. Ta được kết quả: HELLOWORLD → HE LX OW OR LD.
- Thông điệp "HELLOWORLD" được mã hoá như sau: HE → GY, LX → IZ, OW → KO, OR → KC, LD → GC. Kết quả: "GYIZKOKCGC".

Mật mã Playfair được coi là một bước tiến so với các phương pháp mã hóa cổ điển khác vì nó tạo ra các quy tắc phức tạp hơn cho việc mã hoá, làm cho quá trình giải mã trở nên khó khăn hơn so với một số phương pháp khác.

1.4.2 Bài tập thực hành:

- Trong package “week_01” tạo class “PlayfairCipher.java”:

```
package week_01;

import java.util.HashSet;
import java.util.Set;

public class PlayfairCipher {

    private char[][] keyMatrix;
    private static final int MATRIX_SIZE = 5;

    public PlayfairCipher(String key) {
        keyMatrix = generateKeyMatrix(key);
    }

    private char[][] generateKeyMatrix(String key) {
        char[][] matrix = new char[MATRIX_SIZE][MATRIX_SIZE];
        Set<Character> used = new HashSet<>();
        StringBuilder keyBuilder = new StringBuilder();

        for (char ch : key.toCharArray()) {
            if (Character.isLetter(ch)) {
                ch = Character.toUpperCase(ch);
                if (ch == 'J') {
                    ch = 'I';
                }
                if (!used.contains(ch)) {
                    keyBuilder.append(ch);
                    used.add(ch);
                }
            }
        }
    }
}
```



```

        for (char ch = 'A'; ch <= 'Z'; ch++) {
            if (ch == 'J') {
                continue;
            }
            if (!used.contains(ch)) {
                keyBuilder.append(ch);
                used.add(ch);
            }
        }

        int index = 0;
        for (int row = 0; row < MATRIX_SIZE; row++) {
            for (int col = 0; col < MATRIX_SIZE; col++) {
                matrix[row][col] = keyBuilder.charAt(index++);
            }
        }

        return matrix;
    }
}

```

```

private String preprocessText(String text, boolean encode) {
    text = text.toUpperCase().replaceAll("[^A-Z]", "").replace("J", "I");
    StringBuilder sb = new StringBuilder(text);
    for (int i = 0; i < sb.length() - 1; i += 2) {
        if (sb.charAt(i) == sb.charAt(i + 1)) {
            sb.insert(i + 1, 'X');
        }
    }
    if (sb.length() % 2 != 0) {
        sb.append('X');
    }
    return sb.toString();
}

public String encrypt(String plaintext) {
    return processText(plaintext, true);
}

```

```

public String decrypt(String ciphertext) {
    StringBuilder plaintext = new StringBuilder();
    ciphertext = ciphertext.toUpperCase().replaceAll("[^A-Z]", "");

    for (int i = 0; i < ciphertext.length(); i += 2) {
        char firstChar = ciphertext.charAt(i);
        char secondChar = ciphertext.charAt(i + 1);

        int[] pos1 = findPosition(firstChar);
        int[] pos2 = findPosition(secondChar);

        if (pos1[0] == pos2[0]) {
            plaintext.append(keyMatrix[pos1[0]][(pos1[1] + 4) % 5]);
            plaintext.append(keyMatrix[pos2[0]][(pos2[1] + 4) % 5]);
        } else if (pos1[1] == pos2[1]) {
            plaintext.append(keyMatrix[(pos1[0] + 4) % 5][pos1[1]]);
            plaintext.append(keyMatrix[(pos2[0] + 4) % 5][pos2[1]]);
        } else {
            plaintext.append(keyMatrix[pos1[0]][pos2[1]]);
            plaintext.append(keyMatrix[pos2[0]][pos1[1]]);
        }
    }

    for (int i = 0; i < plaintext.length() - 1; i++) {
        if (plaintext.charAt(i) == 'X' &&
            plaintext.charAt(i - 1) == plaintext.charAt(i + 1)) {
            plaintext.deleteCharAt(i);
        }
    }

    if (plaintext.charAt(plaintext.length() - 1) == 'X') {
        plaintext.deleteCharAt(plaintext.length() - 1);
    }

    return plaintext.toString();
}

```

```

private String processText(String text, boolean encrypt) {
    text = preprocessText(text, encrypt);
    StringBuilder result = new StringBuilder();

    for (int i = 0; i < text.length(); i += 2) {
        char a = text.charAt(i);
        char b = text.charAt(i + 1);
        int[] posA = findPosition(a);
        int[] posB = findPosition(b);

        if (posA[0] == posB[0]) {
            result.append(keyMatrix[posA[0]][(posA[1]
                + (encrypt ? 1 : 4)) % MATRIX_SIZE]);
            result.append(keyMatrix[posB[0]][(posB[1]
                + (encrypt ? 1 : 4)) % MATRIX_SIZE]);
        } else if (posA[1] == posB[1]) {
            result.append(keyMatrix[(posA[0]
                + (encrypt ? 1 : 4)) % MATRIX_SIZE][posA[1]]);
            result.append(keyMatrix[(posB[0]
                + (encrypt ? 1 : 4)) % MATRIX_SIZE][posB[1]]);
        } else {
            result.append(keyMatrix[posA[0]][posB[1]]);
            result.append(keyMatrix[posB[0]][posA[1]]);
        }
    }

    return result.toString();
}

```

```

private int[] findPosition(char ch) {
    for (int row = 0; row < MATRIX_SIZE; row++) {
        for (int col = 0; col < MATRIX_SIZE; col++) {
            if (keyMatrix[row][col] == ch) {
                return new int[]{row, col};
            }
        }
    }
    return null;
}

```

```

    public String getKeyMatrixAsString() {
        StringBuilder matrixString = new StringBuilder();
        for (int row = 0; row < MATRIX_SIZE; row++) {
            for (int col = 0; col < MATRIX_SIZE; col++) {
                matrixString.append(keyMatrix[row][col]).append(" ");
            }
            matrixString.append("\n");
        }
        return matrixString.toString();
    }
}

```

- Tạo JFrame Form “frm_Playfair.java”:

- Thêm các gói cần thiết và khởi tạo:

```

package week_01;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import javax.swing.JFileChooser;
import javax.swing.JOptionPane;
import javax.swing.filechooser.FileNameExtensionFilter;

public class frm_Playfair extends javax.swing.JFrame {

    private PlayfairCipher playfairCipher;

    public frm_Playfair() {
        initComponents();
    }
}

```

- Phương thức xử lý sự kiện nhấn nút “Generate Key Matrix”:

```
private void btn_generateKeyMatrixActionPerformed(java.awt.event.ActionEvent evt) {
    String key = txt_key.getText();
    if (!key.isEmpty()) {
        playfairCipher = new PlayfairCipher(key);
        String keyMatrixString = playfairCipher.getKeyMatrixAsString();
        txt_keyMatrix.setText(keyMatrixString);
    } else {
        JOptionPane.showMessageDialog(this, "Please enter key to generate key matrix.");
    }
}
```

- Phương thức xử lý sự kiện nhấn nút "Encrypt":

```
private void btn_encryptActionPerformed(java.awt.event.ActionEvent evt) {
    String plaintext = txt_plaintext.getText();
    String key = txt_key.getText();

    if (!plaintext.isEmpty() && !key.isEmpty()) {
        playfairCipher = new PlayfairCipher(key);
        String ciphertext = playfairCipher.encrypt(plaintext);
        JOptionPane.showMessageDialog(this, "Encrypted successfully.");
        txt_ciphertext.setText(ciphertext);
        saveToFile(ciphertext);
    } else {
        JOptionPane.showMessageDialog(this, "Please enter plaintext and key.");
    }
}
```

- Phương thức xử lý sự kiện nhấn nút "Decrypt":

```
private void btn_decryptActionPerformed(java.awt.event.ActionEvent evt) {
    String ciphertext = txt_ciphertext.getText();
    String key = txt_key.getText();

    if (!ciphertext.isEmpty() && !key.isEmpty()) {
        playfairCipher = new PlayfairCipher(key);
        String plaintext = playfairCipher.decrypt(ciphertext);
        JOptionPane.showMessageDialog(this, "Decrypted successfully.");
        txt_plaintext.setText(plaintext);
    } else {
        JOptionPane.showMessageDialog(this, "Please enter ciphertext and key.");
    }
}
```

- Phương thức xử lý sự kiện nhấn nút "Open Ciphertext File":


```

private void btn_openfileActionPerformed(java.awt.event.ActionEvent evt) {
    JFileChooser fileChooser = new JFileChooser();
    fileChooser.setFileFilter(new FileNameExtensionFilter("Text Files", "txt"));

    int userSelection = fileChooser.showOpenDialog(this);
    if (userSelection == JFileChooser.APPROVE_OPTION) {
        try (BufferedReader bufferedReader =
            new BufferedReader(new FileReader(fileChooser.getSelectedFile()))) {
            JOptionPane.showMessageDialog(this, "File opened successfully.");
            txt_ciphertext.read(bufferedReader, null);
        } catch (IOException e) {
            JOptionPane.showMessageDialog(this, "Error opening file: " + e.getMessage());
        }
    }
}

```

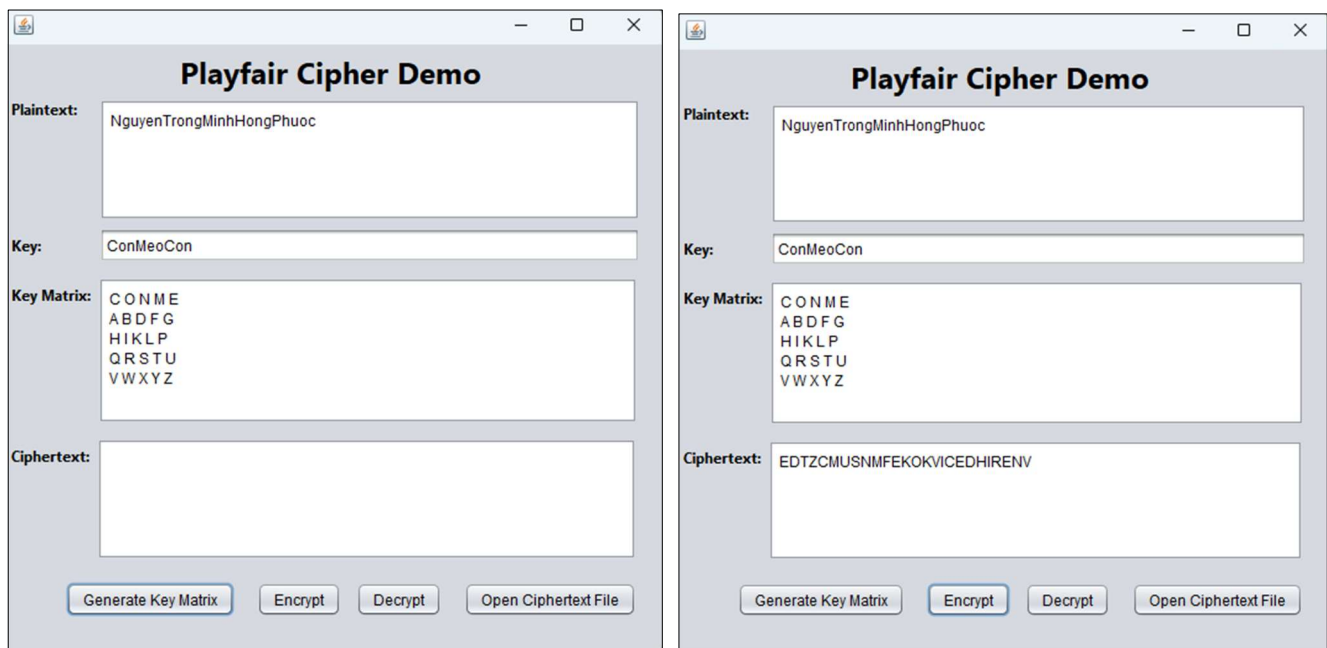
- Phương thức xử lý lưu bản mã vào file:

```

private void saveToFile(String content) {
    JFileChooser fileChooser = new JFileChooser();
    FileNameExtensionFilter filter =
        new FileNameExtensionFilter("Text Files", "txt");
    fileChooser.setFileFilter(filter);
    int userSelection = fileChooser.showSaveDialog(this);
    if (userSelection == JFileChooser.APPROVE_OPTION) {
        try (FileWriter fileWriter =
            new FileWriter(fileChooser.getSelectedFile() + ".txt")) {
            fileWriter.write(content);
        } catch (IOException e) {
            JOptionPane.showMessageDialog(this, "Error saving file: " + e.getMessage());
        }
    }
}

```

- Kiểm tra ứng dụng: Chuột phải tại file cần chạy, chọn "Run File" hoặc nhấn tổ hợp phím "Shift + F6".
- Kiểm tra lần lượt các nút "Generate Key Matrix", "Encrypt", "Decrypt", "Open Ciphertext File". Khi mã hoá thành công sẽ xuất hiện hộp thoại để lưu file chứa bản mã. Tiến hành lưu file, sau đó mở file lên để nhập bản mã từ file vào JTextArea txt_ciphertext.



1.5 MẬT MÃ TRANSPOSITION

1.5.1 Giới thiệu:

Mật mã Transposition là một phương pháp mã hóa thông điệp bằng cách hoán đổi vị trí của các ký tự trong thông điệp ban đầu theo một quy tắc cụ thể. Thay vì thay đổi giá trị của các ký tự, mật mã Transposition tập trung vào việc thay đổi thứ tự xuất hiện của chúng. Có nhiều cách thức để thực hiện mật mã Transposition:

- Đảo ngược toàn bộ bản rõ: Nghĩa là bản rõ được viết theo thứ tự ngược lại từ sau ra trước. Ví dụ:
 - + Bản rõ: SECUREEMAIL → Bản mã: LIAMEERUCES
- Mã hóa theo mẫu hình học: Bản rõ được sắp xếp lại theo một mẫu hình học nào đó, thường là một mảng hoặc ma trận hai chiều. Ví dụ:
 - + Bản rõ: BAOMAT
 - + Chuyển thành ma trận 2x3:

B	A	O
M	A	T

- + Nếu lấy các cột theo thứ tự 2, 3, 1 thì bản mã sẽ là: AAOTBM.

- Đổi chỗ cột: Đổi chỗ các kí tự trong bản rõ thành dạng hình chữ nhật theo cột. Ví dụ:

+ Bản rõ: BAOMATTHUDIEN TU

+ Bản rõ được chuyển thành ma trận 3x5 như sau:

B	M	T	D	N
A	A	H	I	T
O	T	U	E	U

- + Vì có 5 cột nên có thể được sắp xếp theo $5! = 120$ cách khác nhau. Nếu ta chuyển vị các cột theo thứ tự 3, 5, 2, 4, 1 rồi lấy các kí tự theo hàng ngang ta sẽ thu được bản mã: TNMDBHTAIAUUTEO.
- Hoán vị các kí tự của bản rõ theo chu kỳ cố định d: Nếu hàm f là hoán vị của một khối gồm d kí tự thì khóa mã hóa được biểu diễn bởi $K(d, f)$. Ví dụ:

+ Với $d = 5$, hoán vị f của dãy 12345 là 35142.

+ Bản rõ: GROUP. Ta có bảng như sau:

Vị trí ban đầu	Vị trí hoán vị	Nội dung mã hoá	Mã hoá
1	3	G	O
2	5	R	P
3	1	O	G
4	4	U	U
5	2	P	R

Theo bảng trên bản rõ "GROUP" được mã hóa thành "OPGUP".

1.5.2 Bài tập thực hành:

- Trong package "week_01" tạo class "TranspositionCipher.java":

```
package week_01;

import java.util.Arrays;

public class TranspositionCipher {

    public String encrypt(String text, int[] key) {
        int numRows = (int) Math.ceil((double) text.length() / key.length);
        char[][] grid = new char[numRows][key.length];
        for (char[] row : grid) {
            Arrays.fill(row, ' ');
        }

        for (int i = 0; i < text.length(); i++) {
            grid[i / key.length][i % key.length] = text.charAt(i);
        }

        StringBuilder ciphertext = new StringBuilder();
        for (int k : key) {
            for (int row = 0; row < numRows; row++) {
                if (grid[row][k - 1] != ' ') {
                    ciphertext.append(grid[row][k - 1]);
                }
            }
        }

        return ciphertext.toString();
    }

    public String decrypt(String text, int[] key) {
        int numRows = (int) Math.ceil((double) text.length() / key.length);
        char[][] grid = new char[numRows][key.length];
        for (char[] row : grid) {
            Arrays.fill(row, ' ');
        }
    }
}
```

```

int textIndex = 0;
for (int k : key) {
    for (int row = 0; row < numRows; row++) {
        if (textIndex < text.length()) {
            grid[row][k - 1] = text.charAt(textIndex++);
        }
    }
}

StringBuilder plaintext = new StringBuilder();
for (int row = 0; row < numRows; row++) {
    for (int col = 0; col < key.length; col++) {
        if (grid[row][col] != ' ') {
            plaintext.append(grid[row][col]);
        }
    }
}

return plaintext.toString();
}

```

- Tạo JFrame Form “frm_Transposition.java”:

The image shows a Java Swing window titled "Transposition Cipher Demo". The window has a light gray background and a title bar. It contains three text input fields: "Plaintext:", "Key:", and "Ciphertext:". Below these fields are three buttons: "Encrypt", "Decrypt", and "Open Ciphertext File".

- Thêm các gói cần thiết và khởi tạo:

```

package week_01;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Arrays;
import javax.swing.JFileChooser;
import javax.swing.JOptionPane;
import javax.swing.filechooser.FileNameExtensionFilter;

public class frm_Transposition extends javax.swing.JFrame {

    private TranspositionCipher transpositionCipher;

    public frm_Transposition() {
        initComponents();
    }

```

- Phương thức xử lý sự kiện nhấn nút "Encrypt":

```

private void btn_encryptActionPerformed(java.awt.event.ActionEvent evt) {
    String plaintext = txt_plaintext.getText();
    String keyStr = txt_key.getText();

    if (!plaintext.isEmpty() && !keyStr.isEmpty()) {
        int[] key = Arrays.stream(keyStr.split(","))
            .mapToInt(Integer::parseInt)
            .toArray();
        transpositionCipher = new TranspositionCipher();
        String ciphertext = transpositionCipher.encrypt(plaintext, key);
        JOptionPane.showMessageDialog(this, "Encrypted successfully.");
        txt_ciphertext.setText(ciphertext);
        saveToFile(ciphertext);
    } else {
        JOptionPane.showMessageDialog(this, "Please enter plaintext and key.");
    }
}

```

- Phương thức xử lý sự kiện nhấn nút "Decrypt":

```

private void btn_decryptActionPerformed(java.awt.event.ActionEvent evt) {
    String ciphertext = txt_ciphertext.getText();
    String keyStr = txt_key.getText();

    if (!ciphertext.isEmpty() && !keyStr.isEmpty()) {
        int[] key = Arrays.stream(keyStr.split(","))
            .mapToInt(Integer::parseInt)
            .toArray();
        transpositionCipher = new TranspositionCipher();
        String plaintext = transpositionCipher.decrypt(ciphertext, key);
        JOptionPane.showMessageDialog(this, "Decrypted successfully.");
        txt_plaintext.setText(plaintext);
    } else {
        JOptionPane.showMessageDialog(this, "Please enter ciphertext and key.");
    }
}

```

- Phương thức xử lý sự kiện nhấn nút “Open Ciphertext File”:

```

private void btn_openfileActionPerformed(java.awt.event.ActionEvent evt) {
    JFileChooser fileChooser = new JFileChooser();
    fileChooser.setFileFilter(new FileNameExtensionFilter("Text Files", "txt"));

    int userSelection = fileChooser.showOpenDialog(this);
    if (userSelection == JFileChooser.APPROVE_OPTION) {
        try (BufferedReader bufferedReader =
            new BufferedReader(new FileReader(fileChooser.getSelectedFile()))) {
            JOptionPane.showMessageDialog(this, "File opened successfully.");
            txt_ciphertext.read(bufferedReader, null);
        } catch (IOException e) {
            JOptionPane.showMessageDialog(this, "Error opening file: " + e.getMessage());
        }
    }
}

```

- Phương thức xử lý lưu bản mã vào file:

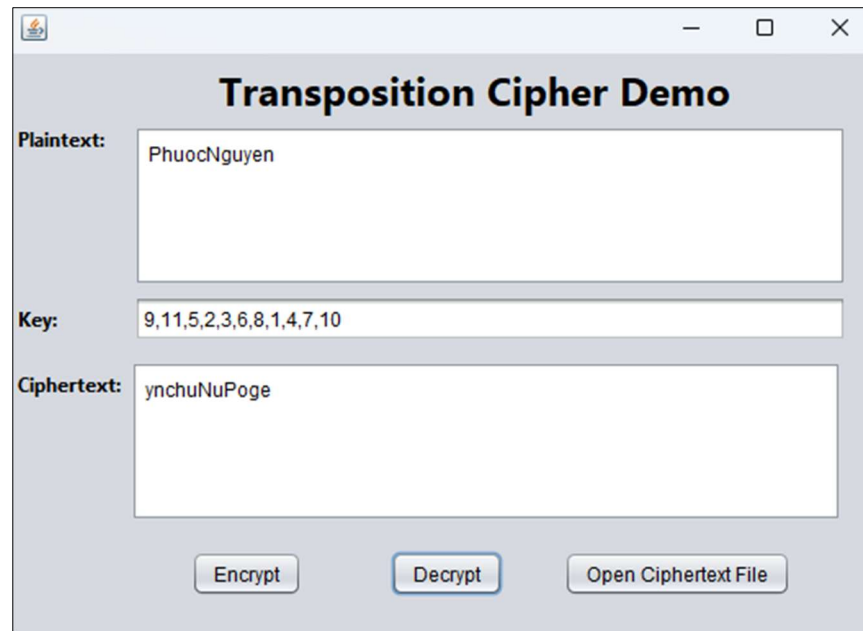
```

private void saveToFile(String content) {
    JFileChooser fileChooser = new JFileChooser();
    FileNameExtensionFilter filter = new FileNameExtensionFilter("Text Files", "txt");
    fileChooser.setFileFilter(filter);
    int userSelection = fileChooser.showSaveDialog(this);
    if (userSelection == JFileChooser.APPROVE_OPTION) {
        try (FileWriter fileWriter =
            new FileWriter(fileChooser.getSelectedFile() + ".txt")) {
            fileWriter.write(content);
        } catch (IOException e) {
            JOptionPane.showMessageDialog(this, "Error saving file: " + e.getMessage());
        }
    }
}

```

- Kiểm tra ứng dụng: Chuột phải tại file cần chạy, chọn “Run File” hoặc nhấn tổ hợp phím “Shift + F6”.

- Kiểm tra lần lượt các nút “Encrypt”, “Decrypt”, “Open Ciphertext File”. Khi mã hoá thành công sẽ xuất hiện hộp thoại để lưu file chứa bản mã. Tiến hành lưu file, sau đó mở file lên để nhập bản mã từ file vào JTextArea txt_ciphertext.



1.6 MẬT MÃ AUTOKEY

1.6.1 Giới thiệu:

Mật mã Autokey là một loại mật mã dùng để mã hóa và giải mã văn bản thông qua việc sử dụng một khóa dài có thể biến đổi và không cần phải trùng khớp với độ dài của văn bản cần mã hóa. Phương pháp này thuộc loại mật mã đơn điệu (substitution cipher), nơi mỗi ký tự trong văn bản gốc được thay thế bằng một ký tự khác dựa trên khóa. Autokey cũng có thể được xem như một sự mở rộng của mật mã Vigenère. Cách thức hoạt động:

1. Khóa: Một chuỗi ký tự (thường là từ hoặc câu) được sử dụng để mã hóa và giải mã văn bản. Khóa này sẽ được sử dụng một cách tuần tự, ký tự sau cùng của văn bản mã sẽ là ký tự đầu tiên của khóa.

2. Mã hóa: Mỗi ký tự của văn bản gốc sẽ được thay thế bằng một ký tự khác dựa trên khóa. Cách thức mã hóa này có thể là theo một bảng chữ cái dịch chuyển (shift) hoặc dựa vào một hàm số học nhất định.

3. Giải mã: Để giải mã, ta sử dụng khóa tương tự như trong quá trình mã hóa. Ký tự mã được đưa vào một quá trình ngược lại để thu được văn bản gốc.

Ví dụ minh họa: Giả sử chúng ta có văn bản gốc là "HELLO" và khóa là "KEY". Quá trình mã hóa và giải mã như sau:

- Mã hóa:

+ Khóa: "KEY".

+ Văn bản gốc: "HELLO".

Đầu tiên, ghép khóa vào đầu văn bản gốc để tạo khóa mở rộng: "KEYHE". Mã hóa từng ký tự:

+ H (vị trí 7 trong bảng chữ cái) + K (vị trí 10) = R (vị trí 18, là ký tự S).

+ E (vị trí 4) + E (vị trí 4) = Y (vị trí 24, là ký tự Y).

+ L (vị trí 11) + Y (vị trí 24) = A (vị trí 1, là ký tự A).

+ L (vị trí 11) + H (vị trí 7) = T (vị trí 20, là ký tự U).

+ O (vị trí 15) + E (vị trí 4) = N (vị trí 14, là ký tự O).

Vậy "HELLO" sau khi được mã hóa với khóa "KEY" sẽ thành "SYAUN".

- Giải mã:

+ Khóa: "KEY"

+ Văn bản mã: "SYAUN"

Đầu tiên, ghép khóa vào đầu văn bản mã để tạo khóa mở rộng: "KEYSY". Giải mã từng ký tự:

+ S (vị trí 18) - K (vị trí 10) = H (vị trí 7, là ký tự H).

+ Y (vị trí 24) - E (vị trí 4) = E (vị trí 4, là ký tự E).

+ A (vị trí 1) - Y (vị trí 24) = L (vị trí 11, là ký tự L).

+ U (vị trí 20) - H (vị trí 7) = L (vị trí 11, là ký tự L).

+ N (vị trí 14) - E (vị trí 4) = O (vị trí 15, là ký tự O).

Vậy "SYAUN" sau khi được giải mã với khóa "KEY" sẽ trở thành "HELLO".

Mật mã Autokey có thể áp dụng với nhiều ngôn ngữ và văn bản khác nhau, tuy nhiên điểm yếu của nó là khóa có thể bị dò tìm nếu độ dài của khóa không đủ dài hoặc có sự lặp lại.

1.6.2 Bài tập thực hành:

- Trong package "week_01" tạo class "AutokeyCipher.java":

```
package week_01;

public class AutokeyCipher {

    public String encrypt(String plaintext, String key) {
        plaintext = plaintext.toUpperCase();
        key = key.toUpperCase();

        StringBuilder ciphertext = new StringBuilder();
        int keyIndex = 0;

        for (int i = 0; i < plaintext.length(); i++) {
            char plainChar = plaintext.charAt(i);

            if (Character.isLetter(plainChar)) {
                char keyChar;

                if (keyIndex < key.length()) {
                    keyChar = key.charAt(keyIndex);
                    keyIndex++;
                } else {
                    keyChar = plaintext.charAt(i - key.length());
                }

                int shift = keyChar - 'A';
                char encryptedChar = (char) ((plainChar - 'A' + shift) % 26 + 'A');
                ciphertext.append(encryptedChar);
            } else {
                ciphertext.append(plainChar);
            }
        }

        return ciphertext.toString();
    }
}
```

```

public String decrypt(String ciphertext, String key) {
    ciphertext = ciphertext.toUpperCase();
    key = key.toUpperCase();

    StringBuilder plaintext = new StringBuilder();
    int keyIndex = 0;

    for (int i = 0; i < ciphertext.length(); i++) {
        char cipherChar = ciphertext.charAt(i);

        if (Character.isLetter(cipherChar)) {
            char keyChar;

            if (keyIndex < key.length()) {
                keyChar = key.charAt(keyIndex);
                keyIndex++;
            } else {
                keyChar = plaintext.charAt(i - key.length());
            }

            int shift = keyChar - 'A';
            char decryptedChar = (char) ((cipherChar - 'A' - shift + 26) % 26 + 'A');
            plaintext.append(decryptedChar);
        } else {
            plaintext.append(cipherChar);
        }
    }

    return plaintext.toString();
}

```

- Tạo JFrame Form “frm_Autokey.java”:

The image shows a Java Swing window titled "Autokey Cipher Demo". The window has a light gray background and a blue border. It contains three text input fields with labels "Plaintext:", "Key:", and "Ciphertext:". Below the input fields are three buttons: "Encrypt", "Decrypt", and "Open Ciphertext File".

- Thêm các gói cần thiết và khởi tạo:

```

package week_01;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import javax.swing.JFileChooser;
import javax.swing.JOptionPane;
import javax.swing.filechooser.FileNameExtensionFilter;

public class frm_Autokey extends javax.swing.JFrame {

    private AutokeyCipher autokeyCipher;

    public frm_Autokey() {
        initComponents();
        autokeyCipher = new AutokeyCipher();
    }

```

- Phương thức xử lý sự kiện nhấn nút "Encrypt":

```

private void btn_encryptActionPerformed(java.awt.event.ActionEvent evt) {
    String plaintext = txt_plaintext.getText();
    String key = txt_key.getText();

    if (!plaintext.isEmpty() && !key.isEmpty()) {

        String ciphertext = autokeyCipher.encrypt(plaintext, key);
        JOptionPane.showMessageDialog(this, "Encrypted successfully.");
        txt_ciphertext.setText(ciphertext);
        saveToFile(ciphertext);
    } else {
        JOptionPane.showMessageDialog(this, "Please enter plaintext and key.");
    }
}

```

- Phương thức xử lý sự kiện nhấn nút "Decrypt":

```

private void btn_decryptActionPerformed(java.awt.event.ActionEvent evt) {
    String ciphertext = txt_ciphertext.getText();
    String key = txt_key.getText();

    if (!ciphertext.isEmpty() && !key.isEmpty()) {
        String plaintext = autokeyCipher.decrypt(ciphertext, key);
        JOptionPane.showMessageDialog(this, "Decrypted successfully.");
        txt_plaintext.setText(plaintext);
    } else {
        JOptionPane.showMessageDialog(this, "Please enter ciphertext and key.");
    }
}

```

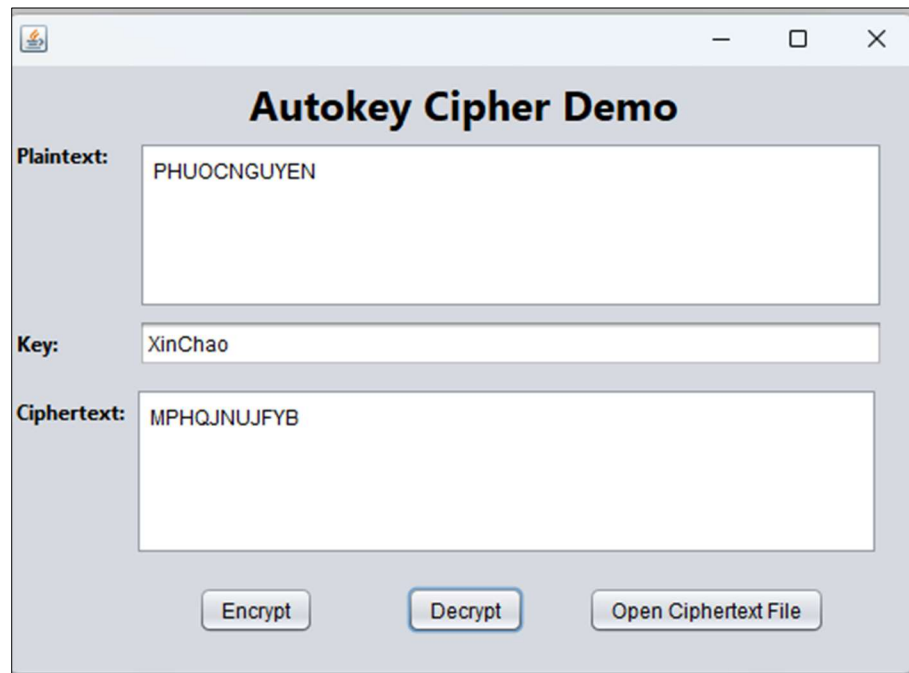
- Phương thức xử lý sự kiện nhấn nút “Open Ciphertext File”:

```
private void btn_openfileActionPerformed(java.awt.event.ActionEvent evt) {  
    JFileChooser fileChooser = new JFileChooser();  
    fileChooser.setFileFilter(new FileNameExtensionFilter("Text Files", "txt"));  
  
    int userSelection = fileChooser.showOpenDialog(this);  
    if (userSelection == JFileChooser.APPROVE_OPTION) {  
        try (BufferedReader bufferedReader =  
            new BufferedReader(new FileReader(fileChooser.getSelectedFile()))) {  
            JOptionPane.showMessageDialog(this, "File opened successfully.");  
            txt_ciphertext.read(bufferedReader, null);  
        } catch (IOException e) {  
            JOptionPane.showMessageDialog(this, "Error opening file: " + e.getMessage());  
        }  
    }  
}
```

- Phương thức xử lý lưu bản mã vào file:

```
private void saveToFile(String content) {  
    JFileChooser fileChooser = new JFileChooser();  
    FileNameExtensionFilter filter = new FileNameExtensionFilter("Text Files", "txt");  
    fileChooser.setFileFilter(filter);  
    int userSelection = fileChooser.showSaveDialog(this);  
    if (userSelection == JFileChooser.APPROVE_OPTION) {  
        try (FileWriter fileWriter =  
            new FileWriter(fileChooser.getSelectedFile() + ".txt")) {  
            JOptionPane.showMessageDialog(this, "File saved successfully.");  
            fileWriter.write(content);  
        } catch (IOException e) {  
            JOptionPane.showMessageDialog(this, "Error saving file: " + e.getMessage());  
        }  
    }  
}
```

- Kiểm tra ứng dụng: Chuột phải tại file cần chạy, chọn “Run File” hoặc nhấn tổ hợp phím “Shift + F6”.
- Kiểm tra lần lượt các nút “Encrypt”, “Decrypt”, “Open Ciphertext File”. Khi mã hoá thành công sẽ xuất hiện hộp thoại để lưu file chứa bản mã. Tiến hành lưu file, sau đó mở file lên để nhập bản mã từ file vào JTextArea txt_ciphertext.



1.7 BÀI TẬP NÂNG CAO

- ❖ **Bài tập 01:** Viết chương trình tổng hợp các mật mã đối xứng cổ điển được trình bày phía trên. Chương trình có giao diện cho phép lựa chọn mật mã cần dùng (tham khảo các control trong Swing). Lưu ý: Đối với các mật mã có bước trung gian (tính toán bảng mã khoá,...) thì thực hiện dưới nền, không cần hiển thị trên giao diện.
- ❖ **Bài tập 02:** Tìm hiểu về các phương pháp tấn công phổ biến lên các hệ mật mã đối xứng cổ điển.